

Proposta Simplificada: Banco de Dados para Urna Eletrônica (MySQL Básico)

Usar um cenário real e motivador (urna eletrônica) para praticar **o básico de SQL no MySQL**: criar tabelas, chaves primárias e estrangeiras, inserir dados e fazer consultas com `SELECT`, `JOIN`, `GROUP BY` e `COUNT`.

"Vamos criar o banco de dados que guarda os **candidatos**, os **eleitores** e os **votos** de uma eleição simples para prefeito."

Modelo Conceitual (bem enxuto)

PARTIDO —< CANDIDATO —< VOTO >— ELEITOR

- Um **partido** tem vários **candidatos**.
- Um **eleitor** pode votar (uma vez) em um **candidato**.
- Cada **voto** liga um eleitor a um candidato.

Tabelas no MySQL

1. partido

Tabela Partido

id_partido - INTEIRO, AUTO INCREMENTO CHAVE PRIMARIA
sigla - VARCHAR(10), NÃO PODE NULO E SER UNICO
nome - VARCHAR(10), NÃO PODE NULO

2. candidato

CREATE TABLE candidato (

TABELA CANDIDATO

id_candidato - INTEIRO, AUTO INCREMENTO CHAVE PRIMARIA
numero - INTEIRO, NÃO PODE NULO E SER UNICO
nome - VARCHAR(10), NÃO PODE NULO
id_partido - INTEIRO, NÃO PODE NULO

FOREIGN KEY (id_partido) REFERENCES partido(id_partido)

3. eleitor

CREATE TABLE eleitor (

cidade VARCHAR(50)
);

TABELA ELEITOR

id_eleitor - INTEIRO, AUTO INCREMENTO CHAVE PRIMARIA
nome - VARCHAR(10), NÃO PODE NULO
titulo - VARCHAR(20), NÃO PODE NULO E SER UNICO
cidade - VARCHAR(50)

4. voto

Tabela VOTO

id_voto - INTEIRO, AUTO INCREMENTO CHAVE PRIMARIA
id_eleitor - INTEIRO, NÃO PODE NULO e SER UNICO -- *UNIQUE garante 1 voto por eleitor*
or
id_candidato - INTEIRO, NÃO PODE NULO
data_hora - DATETIME DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (id_eleitor) REFERENCES eleitor(id_eleitor),
FOREIGN KEY (id_candidato) REFERENCES candidato(id_candidato)

Ponto didático importante: o UNIQUE em id_eleitor ensina, na prática, como impedir voto duplo — sem precisar falar de criptografia.

Inserindo Dados de Teste

EXEMPLOS DE COMO INSERIR DADOS

```
INSERT INTO partido (sigla, nome) VALUES  
( 'ABC', 'Partido Alpha'),  
( 'XYZ', 'Partido Beta');
```

```
INSERT INTO candidato (nome, numero, id_partido) VALUES  
( 'Ana Souza',    10, 1),  
( 'Bruno Lima',   20, 1),  
( 'Carla Mendes', 30, 2);
```

```
INSERT INTO eleitor (nome, titulo, cidade) VALUES  
( 'João',  '111', 'São Paulo'),  
( 'Maria', '222', 'São Paulo'),  
( 'Pedro', '333', 'Campinas'),  
( 'Luiza', '444', 'Campinas');
```

Registrando Votos

EXEMPLO DE COMO REGISTRAR VOTOS

```
INSERT INTO voto (id_eleitor, id_candidato) VALUES  
(1, 1),  -- João votou na Ana  
(2, 3),  -- Maria votou na Carla  
(3, 1),  -- Pedro votou na Ana  
(4, 2);  -- Luiza votou no Bruno
```

Testando a regra do voto único:

```
sql  
INSERT INTO voto (id_eleitor, id_candidato) VALUES (1, 2);  
-- ✗ ERRO: Duplicate entry '1' for key 'id_eleitor'
```

Excelente para mostrar o que é **constraint** na prática.

Consultas que VOCES Devem Praticar

Listar todos os candidatos com seu partido

```
SELECT c.nome AS candidato, p.sigla AS partido, c.numero  
FROM candidato c  
JOIN partido p ON c.id_partido = p.id_partido;
```

Quantos votos cada candidato recebeu

```
SELECT c.nome AS candidato, COUNT(v.id_voto) AS total_votos  
FROM candidato c  
LEFT JOIN voto v ON c.id_candidato = v.id_candidato  
GROUP BY c.id_candidato  
ORDER BY total_votos DESC;
```

Quem ganhou a eleição

```
SELECT c.nome AS vencedor, COUNT(*) AS votos  
FROM voto v  
JOIN candidato c ON v.id_candidato = c.id_candidato  
GROUP BY c.id_candidato  
ORDER BY votos DESC  
LIMIT 1;
```

Quem ainda não votou

```
SELECT nome FROM eleitor  
WHERE id_eleitor NOT IN (SELECT id_eleitor FROM voto);
```

Total de votos por partido

```
SELECT p.sigla, COUNT(*) AS votos
FROM voto v
JOIN candidato c ON v.id_candidato = c.id_candidato
JOIN partido p ON c.id_partido = p.id_partido
GROUP BY p.id_partido;
```